TP1 – Sistema de Gestão de Biblioteca Universitária

Aluno: Leonardo Salerno Arantes

Disciplina: Projeto de Bloco: Desenvolvimento Back-End [26E4_5]

Trilha escolhida: Java 21

Data: 24/10/2026

1. Sobre o projeto

O objetivo é criar um sistema para ajudar a biblioteca de uma universidade a controlar seus livros, exemplares, membros, empréstimos, devoluções, reservas e multas. Neste primeiro trabalho, a ideia é entender as regras, fazer o diagrama de casos de uso e começar um programa simples em Java.

2. Pessoas que vão usar o sistema

• Membro: aluno, professor ou funcionário que pode consultar livros, pedir empréstimos e devoluções e fazer reservas.

• Bibliotecário/administrador: pessoa que cadastra e altera os dados, confirma empréstimos e devoluções e cuida das multas e reservas.

3. Dados principais

• Livro: título, autor e ISBN.

• Exemplar: código de patrimônio, estado de conservação e livro ao qual pertence.

• Membro: nome, matrícula e tipo de membro.

• Empréstimo: membro, exemplar, retirada, prazo e devolução efetiva.

• Reserva: livro, membro e data da reserva.

• Multa: membro, empréstimo, valor e situação de pagamento.

4. Requisitos funcionais (o que o sistema deve fazer)

RF01: Permitir consultar livros pelo título, autor ou ISBN.

RF02: Mostrar quais exemplares existem e se estão disponíveis.

RF03: Permitir ao bibliotecário cadastrar, alterar e remover livros.

RF04: Permitir ao bibliotecário cadastrar e atualizar exemplares e seu estado.

RF05: Permitir ao bibliotecário cadastrar, alterar e remover membros.

RF06: Verificar se o membro tem cadastro válido.

RF07: Permitir que o membro solicite um empréstimo.

RF08: Permitir ao bibliotecário confirmar o empréstimo após as verificações.

RF09: Registrar a data de retirada e o prazo de devolução.

RF10: Permitir que o membro solicite a devolução.

RF11: Permitir ao bibliotecário concluir a devolução e registrar a data.

RF12: Calcular multa quando houver atraso.

RF13: Permitir ao bibliotecário registrar multas e controlar os pagamentos.

RF14: Permitir reservas de livros quando a regra de reserva for atendida.

RF15: Permitir ao bibliotecário consultar e gerenciar reservas.

5. Requisitos não funcionais (como o sistema deve funcionar)

RNF01: Usar Java 21 durante o projeto.

RNF02: Usar IntelliJ IDEA Community como ambiente de desenvolvimento.

RNF03: Ter mensagens claras para orientar o usuário.

RNF04: Tratar entradas inválidas sem encerrar o programa de forma inesperada.

RNF05: Restringir funções administrativas ao bibliotecário quando houver controle de acesso.

RNF06: Manter os dados dos empréstimos e devoluções consistentes quando houver banco de dados.

RNF07: No TP1, funcionar pelo terminal, sem banco de dados e sem interface web.

RNF08: Na futura versão web, buscar um tempo de resposta adequado para consultas (meta a definir com o professor).

6. Regras de negócio

RN01: Cada empréstimo dura 14 dias corridos.

RN02: O membro pode ter no máximo 3 empréstimos ativos.

RN03: Só pode emprestar um exemplar disponível e em bom estado.

RN04: O mesmo exemplar não pode ter dois empréstimos ativos.

RN05: O atraso começa depois do prazo e termina na devolução efetiva.

RN06: Se houver atraso, deve ser criada uma multa vinculada ao empréstimo e ao membro.

RN07: A devolução encerra o empréstimo e registra sua data.

RN08: A reserva é do livro, não de um exemplar específico.

RN09: Só é possível reservar um livro se pelo menos um exemplar dele estiver emprestado; não se reserva quando todos estão disponíveis.

RN10: Um exemplar danificado só volta a ser emprestado depois de regularizado.

RN11: Somente o bibliotecário pode alterar cadastros e gerenciar multas.

RN12: O membro precisa de cadastro válido para empréstimos e reservas.

7. Casos de uso (descrição dos cenários)

Caso 1 – Consultar acervo

Ator: Membro ou bibliotecário.

Fluxo principal: O usuário pesquisa por um livro. O sistema encontra os resultados e mostra os exemplares e a disponibilidade.

Fluxo alternativo: Se não encontrar o livro, o sistema informa que não há resultados. Se os exemplares estiverem indisponíveis, mostra essa situação.

Caso 2 – Processar empréstimo

Ator: Bibliotecário (a pedido de um membro).

Fluxo principal: O bibliotecário informa o membro e o exemplar. O sistema verifica cadastro, limite de empréstimos, disponibilidade e conservação. Se estiver tudo certo, registra a retirada e o vencimento 14 dias depois.

Fluxo alternativo: Se o membro não tiver cadastro válido, já tiver três empréstimos ou o exemplar estiver indisponível/danificado, o sistema recusa e informa o motivo.

Caso 3 – Processar devolução

Ator: Bibliotecário (com entrega pelo membro).

Fluxo principal: O bibliotecário identifica o empréstimo ativo, registra a data de devolução e encerra o empréstimo. Se houver atraso, calcula a multa.

Fluxo alternativo: Se o empréstimo já estiver encerrado ou não existir, o sistema não realiza a devolução. Se a devolução ocorrer no prazo, não há multa.

Caso 4 – Fazer reserva

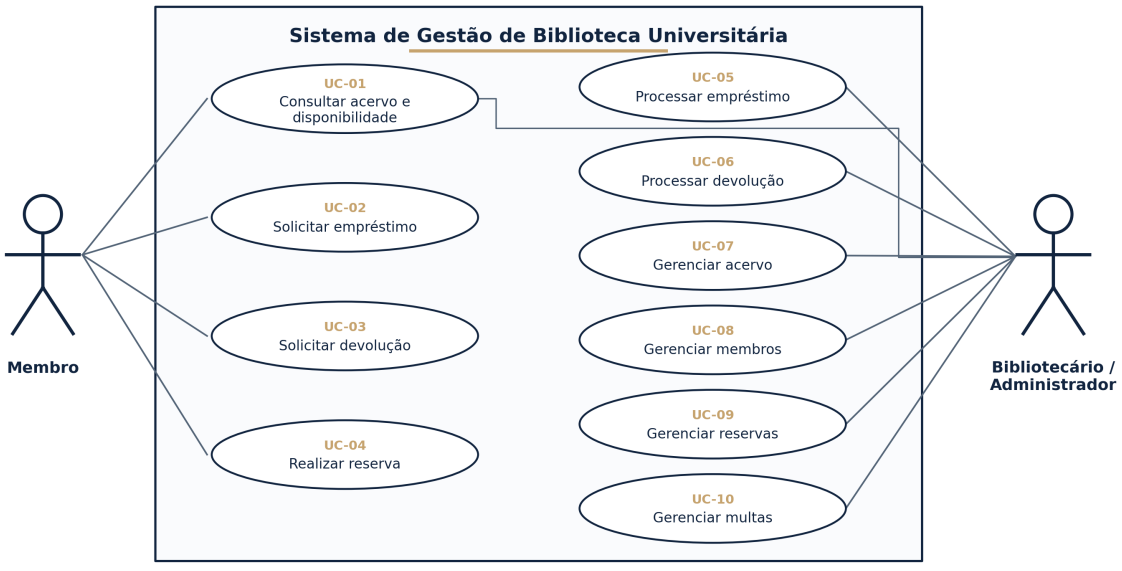
Ator: Membro.

Fluxo principal: O membro escolhe um livro. O sistema verifica cadastro válido e se há pelo menos um exemplar emprestado. Se sim, registra a reserva do livro.

Fluxo alternativo: Se o cadastro for inválido ou nenhum exemplar estiver emprestado, a reserva não é permitida.

8. Diagrama de casos de uso

O desenho mostra as funções do membro e do bibliotecário no sistema. O diagrama representa o sistema completo, mesmo que nem todas as funções estejam implementadas agora.



UML · DIAGRAMA DE CASOS DE USO · ASSOCIAÇÕES ENTRE ATORES E FUNCIONALIDADES

9. Justificativa de escopo e suposições

Neste TP1, foquei nos requisitos, nos casos de uso, no diagrama UML e em um programa simples de terminal em Java. O restante do sistema, como classes, banco de dados, API e autenticação, será desenvolvido nas próximas etapas.

Como o enunciado não informa o valor da multa, considerei R$ 2,00 por dia apenas para testar o cálculo. Também considerei as mesmas regras de empréstimo para todos os tipos de membro. Detalhes sobre renovação e fila de reservas ainda precisam ser definidos.